

Lab 1 - Bit Plane Slicing

June 7, 2025

1 Theory

Bit Plane Slicing involves separating the image into its constituent binary planes. Each pixel in an 8-bit grayscale image is represented by 8 bits. By isolating these bits, we can visualize the contribution of each bit to the overall image. The higher-order bits (most significant bits) contain the majority of the visually significant information, while the lower-order bits (least significant bits) contribute to finer details and noise.

```
[1]: import numpy as np
      from PIL import Image
      import matplotlib.pyplot as plt

[2]: # reading image and converting to gray scale
      img = Image.open('img/prakash.jpg').convert('L')
      # display image
      img

[2]:
```



```
[3]: def bit_slicing_function(input_image):  
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)  
    # convert to numpy array  
    numpy_image = np.array(img)  
  
    fig = plt.figure()  
    fig.set_figheight(100)  
    fig.set_figwidth(100)  
  
    fig.add_subplot(9,1,1)  
    plt.imshow(img, cmap='gray')  
    plt.title('original image')
```

```

#Iterate over each pixel and change pixel value to binary using np.
↪binary_repr() and store it in a list.
lst = []
for i in range(numpy_image.shape[0]):
    for j in range(numpy_image.shape[1]):
        lst.append(np.binary_repr(numpy_image[i][j] ,width=8)) # width = no. of
↪bits

# We have a list of strings where each string represents binary pixel value.
↪To extract bit planes we need to iterate over the strings and store the
↪characters corresponding to bit planes into lists.
# Multiply with 2^(n-1) and reshape to reconstruct the bit image.
eight_bit_img = (np.array([int(i[0]) for i in lst],dtype = np.uint8) * 128).
↪reshape(400,400)
eight_bit_image = Image.fromarray(eight_bit_img)

fig.add_subplot(9,1,2)
plt.imshow(eight_bit_image, cmap='gray')
plt.title('8 bit')

seven_bit_img = (np.array([int(i[1]) for i in lst],dtype = np.uint8) * 64).
↪reshape(400,400)
seven_bit_image = Image.fromarray(seven_bit_img)

fig.add_subplot(9,1,3)
plt.imshow(seven_bit_image, cmap='gray')
plt.title('7 bit')

six_bit_img = (np.array([int(i[2]) for i in lst],dtype = np.uint8) * 32).
↪reshape(400,400)
six_bit_img = Image.fromarray(six_bit_img)

fig.add_subplot(9,1,4)
plt.imshow(six_bit_img, cmap='gray')
plt.title('6 bit')

five_bit_img = (np.array([int(i[3]) for i in lst],dtype = np.uint8) * 16).
↪reshape(400,400)
five_bit_img = Image.fromarray(five_bit_img)

fig.add_subplot(9,1,5)
plt.imshow(five_bit_img, cmap='gray')
plt.title('5 bit')

```

```

four_bit_img = (np.array([int(i[4]) for i in lst],dtype = np.uint8) * 8).
↳reshape(400,400)
four_bit_img = Image.fromarray(four_bit_img)

fig.add_subplot(9,1,6)
plt.imshow(four_bit_img, cmap='gray')
plt.title('4 bit')

three_bit_img = (np.array([int(i[5]) for i in lst],dtype = np.uint8) * 4).
↳reshape(400,400)
three_bit_img = Image.fromarray(three_bit_img)

fig.add_subplot(9,1,7)
plt.imshow(three_bit_img, cmap='gray')
plt.title('3 bit')

two_bit_img = (np.array([int(i[6]) for i in lst],dtype = np.uint8) * 2).
↳reshape(400,400)
two_bit_img = Image.fromarray(two_bit_img)

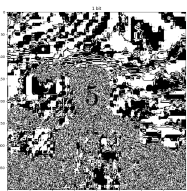
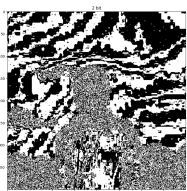
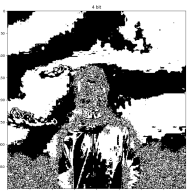
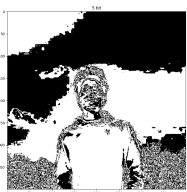
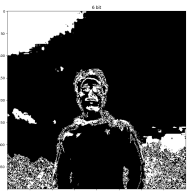
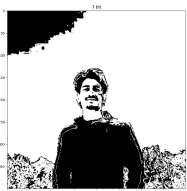
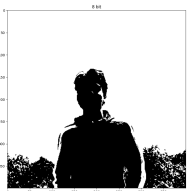
fig.add_subplot(9,1,8)
plt.imshow(two_bit_img, cmap='gray')
plt.title('2 bit')

one_bit_img = (np.array([int(i[7]) for i in lst],dtype = np.uint8) * 1).
↳reshape(400,400)
one_bit_img = Image.fromarray(one_bit_img)

fig.add_subplot(9,1,9)
plt.imshow(one_bit_img, cmap='gray')
plt.title('1 bit')

```

```
[4]: bit_slicing_function(img)
```



2 Observations

- Higher-order bit planes (e.g., 8th and 7th) retain most of the image's structure and major features.
- Lower-order bit planes (e.g., 2nd and 1st) primarily contribute to finer details and noise.

3 Conclusion

Bit Plane Slicing is a useful technique to analyze the importance of individual bits in image representation. It is particularly helpful in image compression, watermarking, and steganography. Through this lab, we observed that higher-order bits retain most of the image's structure, while lower-order bits add finer details and noise.